

End-to-End CI/CD Pipeline using Jenkins, Docker & AWS

Project Title

End-to-End CI/CD Pipeline using Jenkins, Docker & AWS (Full Stack Application Deployment)

Objective

To build and automate a complete CI/CD pipeline where:

- Code push triggers automatic build.
- Docker images are created and pushed.
- Application is deployed automatically on AWS EC2.

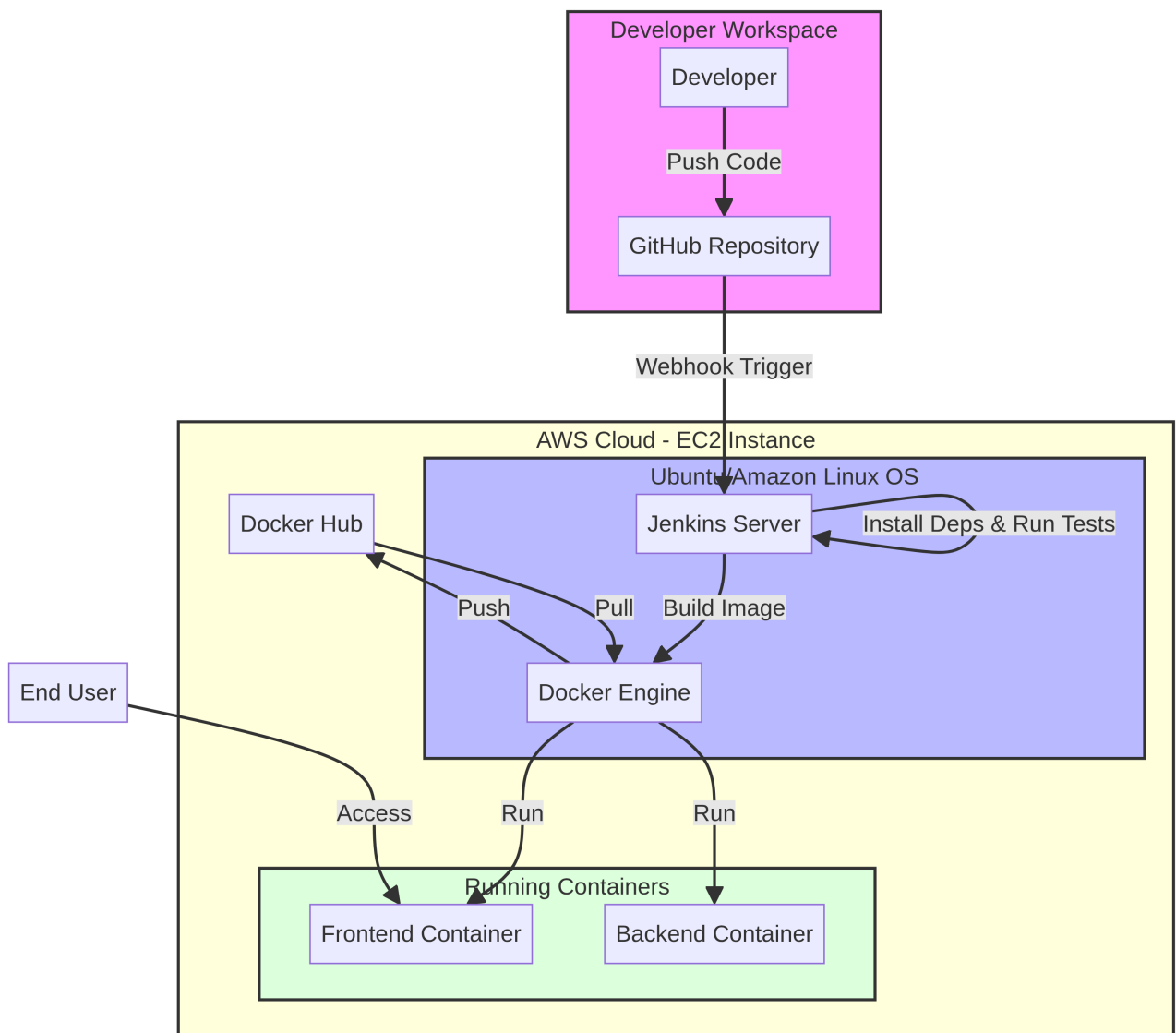
Project Overview

This project demonstrates how to:

- Containerize a full-stack application.
- Automate build and deployment using Jenkins.
- Use Docker for image creation and deployment.
- Host application on Amazon Web Services EC2.

Architecture

The following diagram illustrates the end-to-end CI/CD pipeline architecture:



Flow: GitHub → Jenkins → Docker Build → Docker Hub → EC2 Deployment → Live Application

Technologies Used

Category	Technology	Purpose
CI/CD Automation	Jenkins	Orchestration of pipeline
Containerization	Docker	Image creation & runtime
Cloud Hosting	AWS EC2	Virtual server deployment
Backend	Node.js & Express	Server-side application
Frontend	React + Vite	User interface development
Version Control	GitHub	Source code management

Step-by-Step Implementation

This section outlines the key steps involved in setting up the CI/CD pipeline.

1. System Setup

```
sudo yum update -y
```

2. Clone Project

```
git clone https://github.com/Satyanarayan4434/jenkins-project.git
cd jenkins-project
```

3. Install Docker

```
sudo yum install docker -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```

4. Install Git

```
sudo yum install git -y
git --version
```

5. Install Java (Required for Jenkins)

```
sudo yum install java-21-amazon-corretto -y
java -version
```

6. Install Jenkins

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/redhat-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io.key
sudo yum install jenkins -y
sudo systemctl daemon-reload
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

7. Give Docker Permission to Jenkins

```
sudo usermod -aG docker jenkins
```

8. Unlock Jenkins

Retrieve the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Access Jenkins via browser:

http://<EC2-IP>:8080

Jenkins Configuration

Plugins Installed:

- Git Plugin
- Pipeline Plugin
- Docker Pipeline Plugin
- GitHub Integration

Credentials Added:

- Docker Hub credentials (`dockerhub-creds`)

CI/CD Pipeline Flow

1. Developer pushes code to GitHub.
2. Jenkins detects change (Webhook).
3. Install dependencies.
4. Run tests.
5. Build Docker image.
6. Push image (optional - Docker Hub).
7. Deploy to EC2.
8. Run container.

Jenkins Pipeline (Jenkinsfile)

```
pipeline {
    agent any

    environment {
        CLIENT_IMAGE = "satyanarayan4434/jenkins-client"
        SERVER_IMAGE = "satyanarayan4434/jenkins-server"
        EC2_IP = "YOUR_EC2_IP"
    }

    stages {

        stage('\Build Server Image\') {
            steps {
                dir('\server\') {
                    sh '\docker build -t $SERVER_IMAGE .\'
                }
            }
        }

        stage('\Build Client Image\') {
            steps {
                dir('\client\') {
                    sh '\'\'\
                    docker build \
                    --build-arg VITE_API_URL=http://$EC2_IP:3000 \
                    -t $CLIENT_IMAGE .
                    \''\'\'
                }
            }
        }

        stage('\Push Images\') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: '\dockerhub-creds\' ,
                    usernameVariable: '\USER\' ,
                    passwordVariable: '\PASS\'
                )]) {
                    sh '\'\'\
                    echo $PASS | docker login -u $USER --password-stdin
                    docker push $SERVER_IMAGE
                    docker push $CLIENT_IMAGE
                    \''\'\'
                }
            }
        }
    }
}
```

```

    }
  }
}

stage(\Deploy\') {
  steps {
    sh \'\'\'\''
    docker stop server || true
    docker rm server || true
    docker stop client || true
    docker rm client || true

    docker run -d -p 3000:3000 --name server $SERVER_IMAGE
    docker run -d -p 80:80 --name client $CLIENT_IMAGE
    \'\'\'\''
  }
}
}
}

```

Deployment Output

- **Frontend:** `http://EC2-IP`
- **Backend:** `http://EC2-IP:3000`

Key Learnings

- CI/CD pipeline automation using Jenkins.
- Docker image lifecycle (build → push → deploy).
- Handling environment variables securely (avoiding `.env` push).
- Debugging real-world issues:
 - Docker permission issues.
 - Node version mismatch.
 - Jenkins credential errors.
 - Webhook failures.

Challenges Faced

- Jenkins not starting due to Java version conflicts.
- Docker permission denied for Jenkins user.
- Frontend not connecting due to incorrect API URL configuration.
- Webhook not triggering the pipeline.
- Node version mismatch inside Docker containers.

Future Improvements

- Implement a reverse proxy using Nginx for better traffic management.
- Secure the application with HTTPS using SSL certificates.
- Adopt Blue-Green deployment strategies for zero-downtime updates.
- Migrate to Kubernetes for container orchestration and scalability.

Conclusion

This project successfully demonstrates a real-world DevOps pipeline where manual work is eliminated, deployment is automated, and the application is always up-to-date.